Computação Distribuída

SCTEC – Sistema de Controle de Telescópio Espacial Compartilhado

Entregas 4 e 5

Aluno: Pedro Roberto Fernandes Noronha

RA: 10443434

Professor: Mario

Repositório:

https://github.com/peedroroberto12/Sistema-de-Controle-de-Telesc-pio-Espacial-Compartilhado-SCTEC-

Entrega 4 – A fonte da verdade e o cliente inteligente

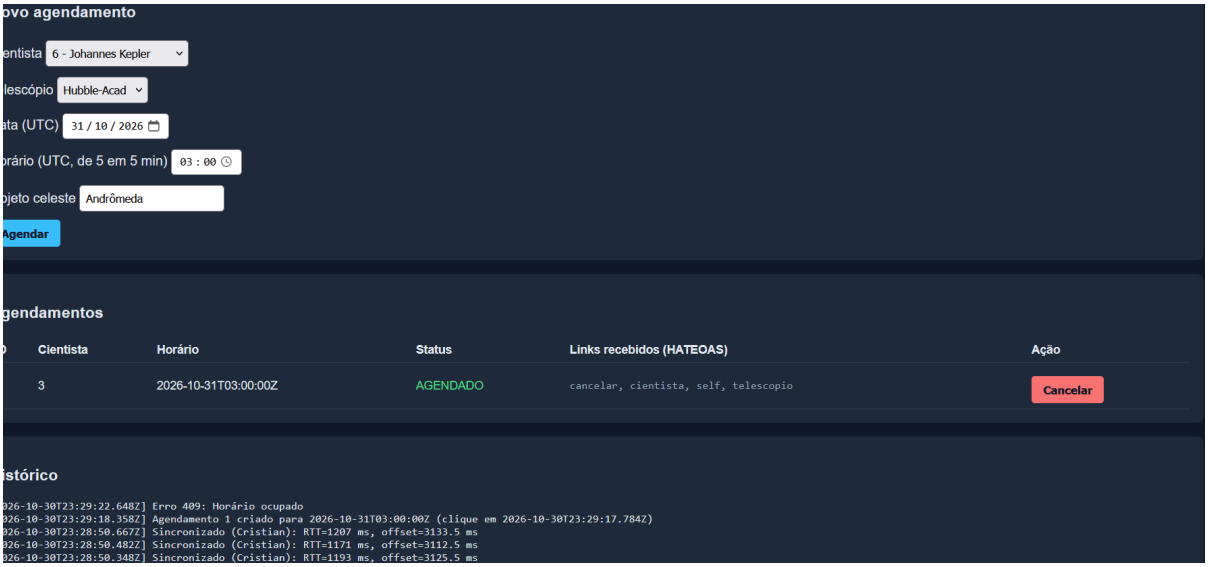
O que foi feito

- GET /time no Flask: devolve a hora oficial do servidor em UTC. Cada chamada gera um log de aplicação.
- DELETE /agendamentos/{id}: cancela o agendamento mudando o status para CANCELADO (o registro não é apagado) e gera o log de auditoria AGENDAMENTO_CANCELADO.
- Página web (static/index.html), acessada em /web: sincroniza o relógio com o algoritmo de Cristian, permite agendar e cancelar.
- HATEOAS no cliente: a página só conhece a raiz da API (/) e pega dela os links das outras rotas. O botão Cancelar só aparece quando o agendamento vem com o link "cancelar", e o clique usa o href e o method desse link.

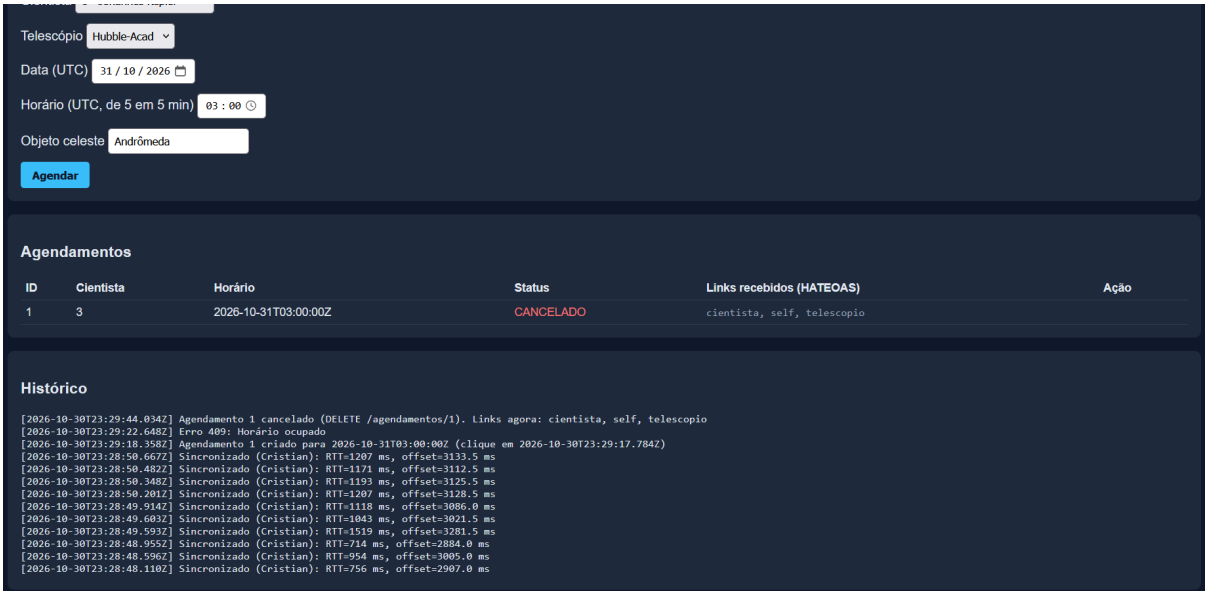
Resultado



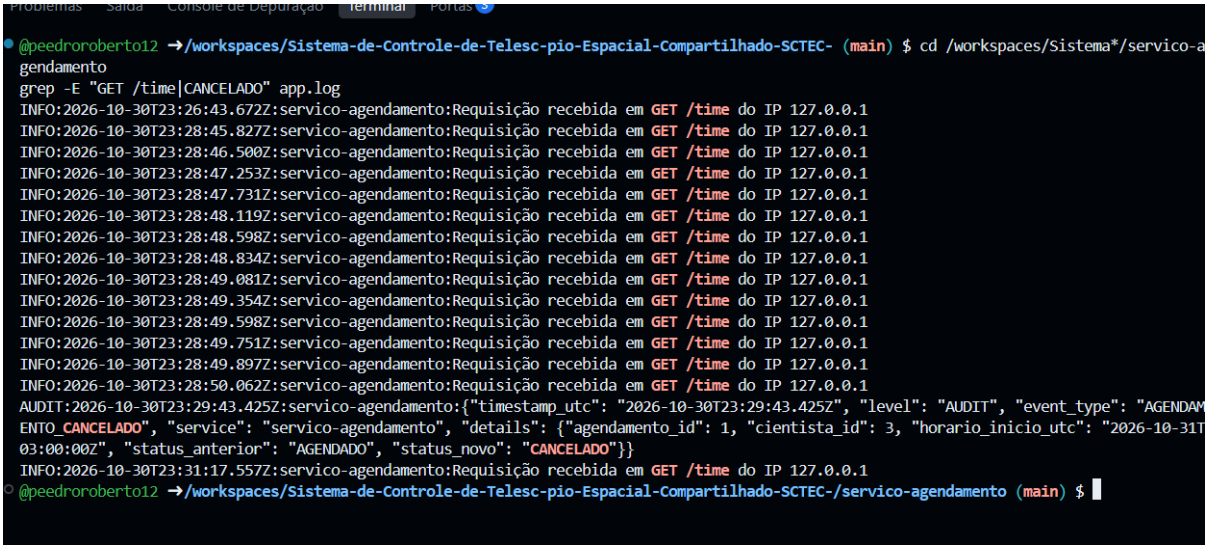
Print 1 – Relógio sincronizado com o servidor (hora do servidor, RTT e offset)



Print 2 – Agendamento criado: o link "cancelar" veio na resposta e o botão aparece. A segunda tentativa no mesmo horário recebeu 409



Print 3 – Depois do cancelamento: status CANCELADO, o link "cancelar" sumiu e o botão também



Print 4 – app.log com as chamadas ao GET /time e a auditoria AGENDAMENTO_CANCELADO

Análise

O relógio do meu PC estava uns 3 segundos atrasado em relação ao servidor (offset de 3133 ms). Sem a sincronização, a hora do clique ia chegar errada no servidor, que é exatamente o problema que o enunciado descreve.

A página usa o algoritmo de Cristian: anota a hora antes de chamar o /time, anota quando a resposta chega e soma metade desse tempo (RTT) na hora do servidor. Com isso ela calcula o offset e corrige o relógio. A hora do clique que vai no agendamento já sai corrigida.

O RTT deu alto, perto de 1,2 s, porque o Codespace passa por um proxy do GitHub. Como o Cristian supõe que a ida e a volta demoram o mesmo tempo, o erro pode ser de até metade do RTT, uns 0,6 s. Pro nosso caso dá pra usar.

No HATEOAS, a página não tem as URLs escritas no código. Ela pega os links na raiz da API, e o botão Cancelar só aparece se o agendamento vier com o link "cancelar". Depois de cancelar, o servidor para de mandar esse link e o botão some sozinho, sem nenhum if no front olhando o status.

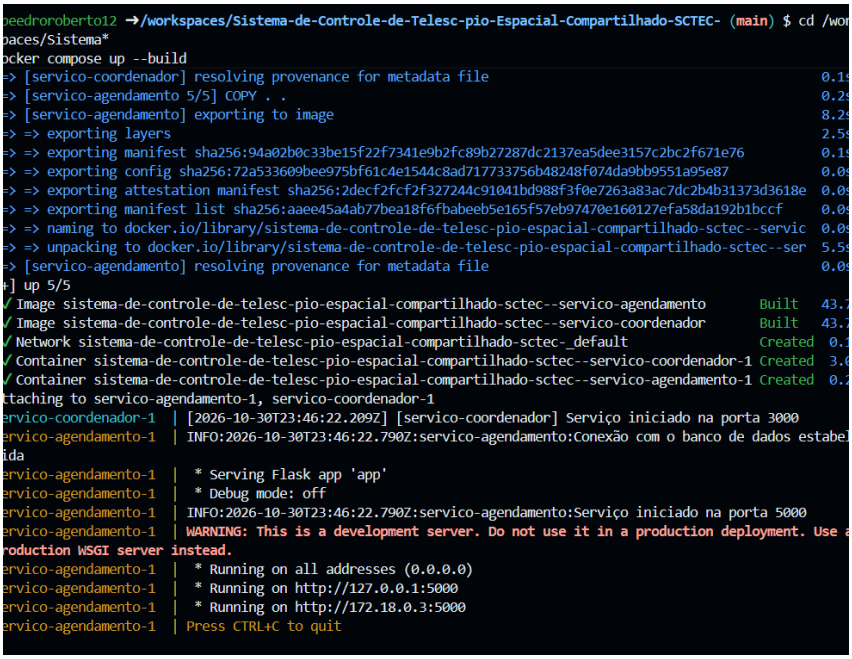
No log aparecem as chamadas ao GET /time e a auditoria AGENDAMENTO_CANCELADO, com o status anterior e o novo. O agendamento não é apagado do banco, então o histórico fica guardado.

Entrega 5 – Orquestração com Docker

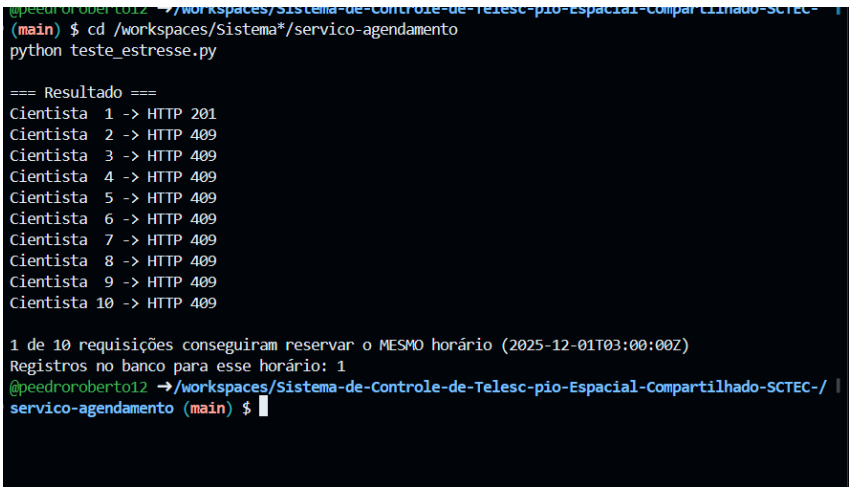
O que foi feito

- Dockerfile do servico-agendamento: imagem python:3.12-slim, instala o requirements.txt e roda o app.py.
- Dockerfile do servico-coordenador: imagem node:20-alpine, instala as dependências e roda o server.js.
- docker-compose.yml na raiz: sobe os dois serviços na mesma rede e expõe as portas 5000 e 3000.
- URL do coordenador: o Flask lê a variável COORDENADOR_URL, e o compose passa http://servico-coordenador:3000. Não foi preciso alterar o código Python.

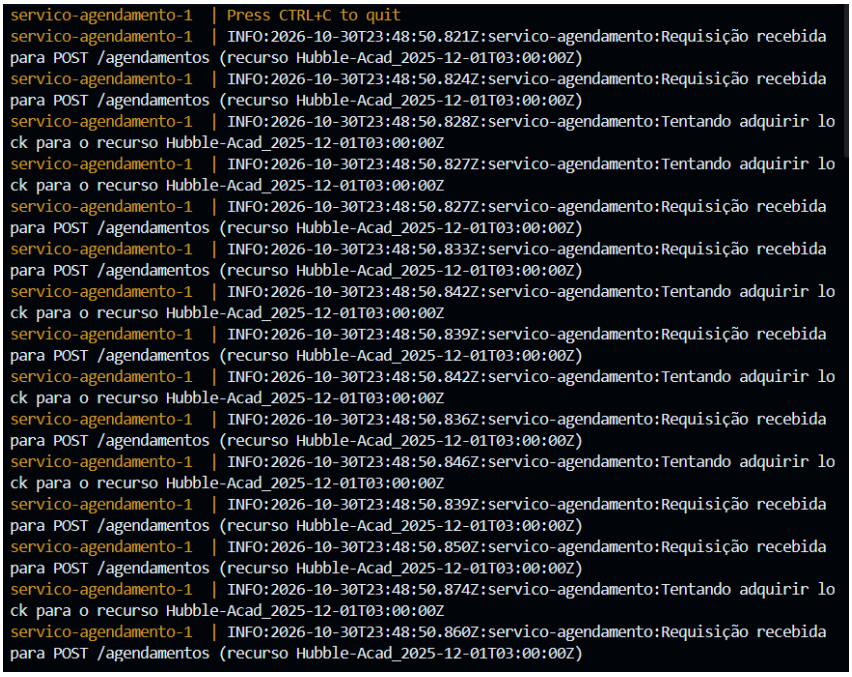
Resultado



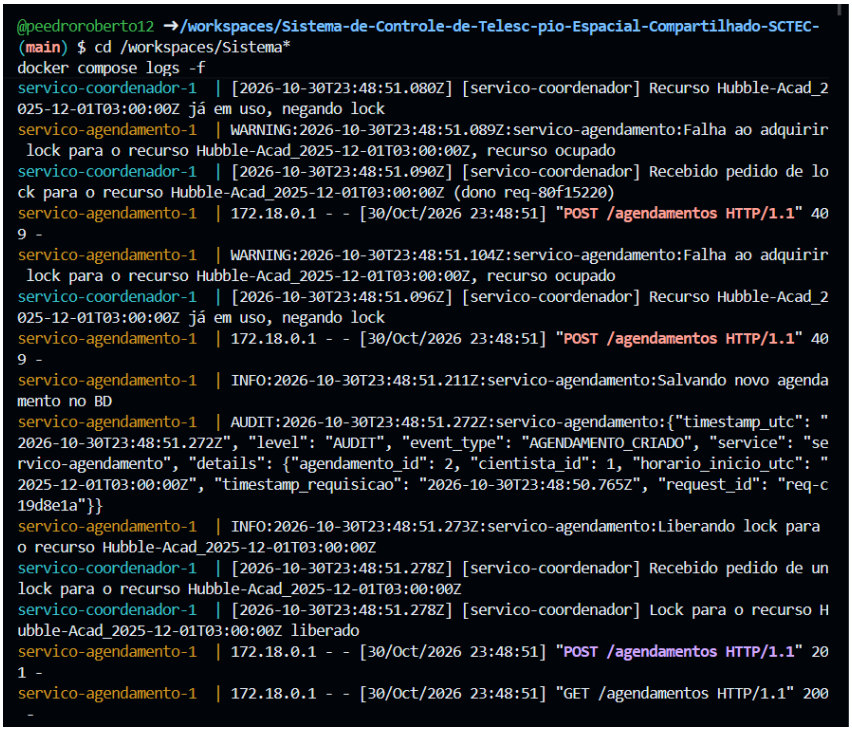
Print 1 – docker compose up --build: imagens criadas e os dois contêineres iniciados



Print 2 – teste_estresse.py contra os contêineres: 1 requisição com 201 e 9 com 409



Print 3 – docker compose logs -f: as 10 requisições chegando ao Flask ao mesmo tempo



Print 4 – docker compose logs -f: logs do Flask e do coordenador intercalados no mesmo fluxo

Análise

Com o docker compose up --build os dois serviços subiram juntos com um comando só, cada um no seu contêiner.

A única mudança foi a URL do coordenador. Dentro do Docker um serviço acha o outro pelo nome, então o Flask passou a usar http://servico-coordenador:3000. Como o código já lia essa URL de uma variável de ambiente, só precisei passar o valor no docker-compose.yml, sem mexer no Python.

O teste de estresse rodando contra os contêineres deu o mesmo resultado da Entrega 3: 1 conseguiu e 9 receberam 409. Ou seja, o lock continua funcionando com os serviços separados.

No docker compose logs -f os logs dos dois serviços aparecem misturados no mesmo terminal. Dá pra ver o coordenador negando o lock e logo em seguida o Flask registrando a falha, e no final o Flask liberando o lock e o coordenador confirmando. Isso facilita muito acompanhar uma requisição passando pelos dois serviços, sem ter que abrir um terminal pra cada um.

Conclusão

Na Entrega 4 deu pra ver na prática que os relógios das máquinas não batem (o meu estava 3 s atrasado) e que o Cristian resolve isso de um jeito simples. O HATEOAS deixou o cliente mais simples, porque é o servidor que diz o que dá pra fazer.

Na Entrega 5 o Docker deixou o sistema fácil de subir em qualquer máquina, e os logs juntos mostram o sistema distribuído funcionando como um todo. Fechando o projeto, cada parte resolveu um problema: o lock resolveu a concorrência, o Cristian resolveu o tempo e os logs deixaram tudo rastreável.